

Thinking OS v2.0.0 Stable

中文日常使用说明书

版本：v2.0.0 Stable

适用对象：日常使用者

定位：AI 思考 / 决策 / 研究 / 规划 / 执行控制系统

01. Thinking OS 到底是什么？

Thinking OS 不是一个“万能提示词”，也不是要求你每次都学习一堆思维模型。

它更像安装在 AI 上面的一个：

思考控制器。

它负责自动判断：

1. 这个问题需不需要认真分析；
2. 应该分析多深；
3. 应该使用哪些思维方法；
4. 你的问题本身有没有问错；
5. 是否需要查最新资料；
6. 应该去哪里找真正的 Source of Truth；
7. 是否需要 Red Team 反驳；
8. 是否需要计算概率、风险、机会成本；
9. 什么时候应该停止分析；
10. 下一步应该做什么。

所以正常使用 Thinking OS 时，你不需要说：

“请使用第一性原理、MECE、OODA、贝叶斯、二阶思维、红队……”

这些事情应该由 Thinking OS 自己决定。

02. 最简单的使用方式

直接正常说话

这是最推荐的方式。

例如你直接说：

我最近想做一个 AI 公众号，但是不知道值不值得投入时间，你帮我看看。

就够了。

你不需要说：

“请启动 Thinking OS。”

也不需要写几十行 Prompt。

Thinking OS 应该自己判断：

- 这是一个项目决策；
- 存在机会成本；
- 需要判断核心假设；
- 需要考虑最小验证；
- 可能需要先验证内容需求，而不是先搭自动化系统。

然后给你结果。

03. Thinking OS 的三种调用方式

方式一：自动调用

推荐度：★★★★★

安装 Thinking OS 后，你就正常提问。

例如：

我想做一个本地 PDF 编辑器，只需要修改姓名和日期，字体尽量保持一致。这个东西应该怎么做？

Thinking OS 会自己判断：

- 有没有 XY Problem；
- 有没有过度工程；
- 最小可行方案是什么；
- 是否真的需要开发完整 PDF 编辑器；
- 下一步应该做什么。

这是日常最推荐的使用方式。

方式二：明确告诉 AI 使用 Thinking OS

当你觉得这个问题比较重要时，可以直接说：

用 Thinking OS 帮我分析一下这个问题。

或者：

用 Thinking OS 判断一下这个项目值不值得做。

如果你的 ChatGPT 界面支持 Skill 的 @ 提及，也可以直接：

@thinking-os 帮我分析这个项目。

适合：

- 重要决策；
- 商业问题；
- 投资问题；
- 项目规划；
- Coding 架构；
- 复杂排错；
- 方案评估；
- 风险判断。

方式三：Thinking OS + 模式指令

这是高级使用方式。

例如：

用 Thinking OS /red 审视一下这个项目。

意思是：

不只是正常分析，还要主动攻击这个方案最容易失败的地方。

或者：

用 Thinking OS /action，我已经决定做了，不要再讨论值不值得，直接告诉我怎么落地。

这样可以强制改变 Thinking OS 当前的工作模式。

04. 日常最重要的 9 个模式指令

不需要全部背下来。

真正日常高频的，大概只有：

`/deep`、`/red`、`/action`、`/simple`

/simple

意思

简单处理，不要想复杂。

例如：

/simple 帮我选一下这两个标题哪个更好。

适合：

- 小问题；
 - 可逆决定；
 - 简单比较；
 - 不值得投入大量分析的问题。
-

/fast

意思

快速给结论。

例如：

/fast 帮我判断一下这个方案最大的三个问题。

Thinking OS 仍然会判断，但会压缩过程。

适合：

- 时间紧；
 - 只想快速判断；
 - 先拿一个方向。
-

/deep

意思

进入深度分析。

例如：

/deep 我准备投入 30 万做这个项目，帮我完整判断值不值得做。

适合：

- 重要决策；
 - 高投入；
 - 多变量；
 - 不确定性高；
 - 做错代价比较大。
-

/max

意思

使用当前问题合理范围内的最高分析强度。

例如：

/max 帮我评估这家公司未来三年的投资逻辑。

注意：

/max 不是“把所有思维模型全部用一遍”。

而是：

在有价值的地方尽可能深入。

/red

意思

开启 Red Team。

主动攻击你的方案。

例如：

/red 我觉得这个商业模式已经很完善了，你专门找它最可能失败的地方。

适合：

- 你已经比较看好某个方案；
 - 怕自己确认偏误；
 - 准备投入大量资金；
 - 准备做不可逆决策；
 - 想做 Premortem。
-

/first

意思

强制第一性原理。

例如：

/first 不考虑行业现在普遍怎么做，从最底层重新分析这个产品有没有必要存在。

适合：

- 行业惯例可能有问题；
 - 怀疑大家都在照抄；
 - 想重新定义问题；
 - 创新问题。
-

/data

意思

提高证据要求。

例如：

/data 帮我判断这个市场到底有没有增长，不要只讲逻辑，优先看真实数据。

适合：

- 投资；
 - 市场研究；
 - 行业研究；
 - 当前事实；
 - 需要证据支撑的问题。
-

/idea

意思

暂时不要太快收敛，多探索可能性。

例如：

/idea 给这个直播活动重新探索玩法，不要局限在我们现在这几个方案。

适合：

- 创意；
- 产品方向；
- 内容策划；
- 新商业模式；
- Brainstorm。

/action

意思

停止讨论战略，进入执行。

例如：

`/action` 这个项目我已经决定做了，不要再讨论做不做，直接拆执行步骤。

这个非常重要。

它等于告诉 Thinking OS：

决策阶段已经结束。

避免 AI 不断跟你说：

“你还需要考虑……”

05. 长期项目的 5 个专用指令

Thinking OS v2.0 还支持长期 Decision State。

/state

查看当前项目判断状态。

例如：

`/state` 给我看看我们现在对这个项目的整体判断。

理想输出包括：

- 当前建议；
- 当前概率；
- 关键假设；

- 已有证据；
 - 最大风险；
 - Kill Criteria；
 - 下一步。
-

/update

有新数据以后更新判断。

例如：

/update 上周我们测试了 30 个用户，其中 9 个愿意付费。更新一下之前的判断。

Thinking OS 应该：

1. 保留以前的判断；
 2. 记录新证据；
 3. 判断影响哪些假设；
 4. 更新概率；
 5. 说明为什么变了。
-

/review

正式复盘。

例如：

/review 这个项目已经运行一个月了，重新审视一下还值不值得继续。

适合：

- 月度复盘；
 - 阶段复盘；
 - 项目里程碑；
 - 投资 Thesis 更新。
-

/killcheck

专门判断该不该停止。

例如：

/killcheck 结合现在的数据，看看是不是已经达到我们之前设定的停止条件。

`/journal`

查看判断变化记录。

例如：

`/journal` 给我看看过去三个月我们对这个项目的判断是怎么变化的。

06. 最推荐的万能提示词格式

虽然你完全可以直接说人话，但是碰到重要问题时，我推荐这个结构：

问题 + 目标 + 已知事实 + 约束 + 你想让我产出的东西

例如：

我现在想做一个 AI 公众号。
目标是半年内做到稳定获客。
我每天能投入 2 小时，预算 5000 元。
目前还没有验证具体内容方向。
帮我判断应该怎么启动，重点看我是不是把事情想复杂了，最后给我最小可执行方案。

这已经是非常好的 Prompt。

07. Thinking OS 推荐 Prompt 模板

A. 做一个重要决定

我现在要决定【事情】。
我的目标是【目标】。
已知情况是【事实】。
约束是【资金/时间/资源】。
帮我用 Thinking OS 判断真正的问题是什么、有哪些选项、各自风险收益是什么，最后给明确建议和下一步行动。

例如：

我现在要决定要不要投入 20 万开发这个产品。目标是一年内做到盈利。目前只有几个朋友说愿意用，没有真实付费数据。帮我判断值不值得做，最后给明确建议。

B. 怀疑自己的方向错了

我现在认为【我的判断】。

但我担心自己存在确认偏误。

`/red` 帮我找这个判断最可能错在哪里，以及什么证据能够推翻它。

C. 从第一性原理重新分析

`/first`

我们现在行业通常做法是【做法】。

不要默认这个做法合理，从目标和底层约束重新分析，看看有没有更简单的路径。

D. 做项目规划

我要完成【目标】。

当前资源是【资源】。

时间是【时间】。

帮我拆成最少必要模块，找关键路径和当前最大瓶颈，然后给第一个可以马上执行的动作。

E. 项目已经决定做

`/action`

这个方向已经确定，不再讨论做不做。

目标是【目标】。

当前状态是【状态】。

直接给执行路径、优先级、依赖、验收标准和第一步。

F. 找问题原因

现在出现的问题是【现象】。

已知信息是【信息】。

不要直接猜原因。

帮我列最可能的原因，并告诉我先做哪个测试最容易区分这些原因。

这个特别适合 Coding、运营数据异常、商业问题。

G. 评估一个已有方案

这是我的方案：【方案】。

不需要重新从零设计。

先判断方案整体是否成立，再找关键假设、最大风险和最薄弱环节，最后告诉我应该保留、修改还是推翻。

H. 做市场 / 行业研究

`/data`

我要判断【市场/行业问题】。

优先使用当前、可靠、接近 Source of Truth 的数据。

区分事实、推断和未知，不要用资料数量代替证据质量。

最后告诉我这些证据是否足以改变决策。

I. Coding / GitHub / Repo

根据实际代码判断【问题】。

不要根据一般经验猜项目实现。

先看真实 Repo / 配置 / CI，再判断问题和修改方案。

例如：

根据实际代码判断这个项目有没有必要改成微服务，不要先讲微服务理论。

J. 文件分析

上传文件，然后直接说：

基于我上传的文件真实内容分析，不要根据文件名猜。帮我找出【问题】。

例如：

基于这个 PDF 真实内容，把关键条款、风险和需要我做决定的地方提出来。

08. 你其实不需要写“专业 Prompt”

这是 Thinking OS 很重要的设计原则。

比如你可以直接语音说：

我现在有点纠结，我想搞个网站，然后又想做 App，但是我其实还不知道有没有人用。然后我觉得网站可能比较简单，你看看我到底应该怎么弄？

这完全可以。

Thinking OS 应该自己整理成：

- 目标；
- 已知事实；
- 未知；

- 假设；
- 决策；
- 最小验证。

所以：

不要因为用了 Thinking OS，反而逼自己学习“提示词工程”。

它本来的目的就是降低这个负担。

09. 什么情况下最好主动写 `/deep`

以下问题我建议主动加：

`/deep`

例如：

- 几十万以上资金投入；
- 创业方向；
- 长期投资；
- 重大技术架构；
- 职业选择；
- 高成本项目；
- 不可逆决策；
- 多方利益冲突；
- 不确定性很高的战略问题。

示例：

`/deep` 我准备投入 50 万做这件事。在决定以前完整审视一次。

10. 什么情况下最好主动写 `/red`

当你发现自己已经非常喜欢一个方案的时候。

尤其应该 Red Team。

例如：

`/red` 我现在已经很想做这个项目了，所以你不要顺着我。站在最强反对者角度审视一次。

这个用法非常好。

11. 什么情况下应该用 `/action`

当你发现 AI 又开始讨论：

“要不要做”

但你已经决定了。

直接：

`/action`

例如：

``/action`，不需要再讨论这个方案值不值得。现在目标就是 30 天上线，给我执行计划。

12. 什么情况下应该用 `/simple`

当你自己都知道：

这事根本没那么重要。

例如：

`/simple`，这两个域名选一个就行，不要给我写商业战略。

Thinking OS 应该立刻降档。

13. 多个指令可以组合吗？

可以，但不要乱叠。

例如：

`/deep /red`

意思：

深度分析 + Red Team。

适合重要方案。

例如：

`/data /deep`

意思：

深度分析，并提高证据要求。

适合投资、商业研究。

例如：

`/first /idea`

意思：

从底层重新定义问题，并扩大探索空间。

适合创新问题。

但是不建议：

`/max /deep /red /first /data /idea /action`

全部一起写。

这相当于同时踩油门、刹车、倒车。

Thinking OS 本来就是为了自动选择这些模式。

14. 最推荐的使用原则

原则一：默认先什么都不写

先直接问。

只有结果明显不符合你的需求，再加模式。

原则二：告诉 AI 真实目标

比“使用什么模型”更重要。

不要只说：

帮我分析这个项目。

最好说：

我真正的目标是半年内验证它能不能稳定赚钱。

原则三：把真实约束告诉 AI

例如：

- 时间；
- 预算；
- 人数；
- 技术能力；
- 已有资源；
- 不能接受的风险。

因为没有约束的最佳方案，经常不是现实中的最佳方案。

原则四：事实和想法尽量分开

例如：

事实：现在有 100 个注册用户，3 个付费。

我的判断：我觉得主要问题是价格。

你帮我判断这个判断是否成立。

这比直接说：

用户不付费就是因为太贵。

好得多。

15. 一个非常强的万能句式

如果你不知道怎么问，可以直接用这句：

我现在的情况是：【把事情说出来】。我的真正目标是：【目标】。你用 Thinking OS 自己判断需要多深、要不要查资料、要不要挑战我的前提，不要为了分析而分析，最后给我明确结论和下一步动作。

基本可以覆盖大部分复杂任务。

16. 想让 Thinking OS 主动挑战你的问题，可以这样说

不要默认我的问题问对了。如果我在解决错误的问题，先告诉我真正应该解决什么。

这是一个非常推荐的附加句。

例如：

我想优化一天生产 20 条短视频的流程。不要默认“一天20条”这个目标本身正确，如果我问错问题先纠正我。

Thinking OS 可能最后发现：

真正需要优化的是：

每单位时间产生有效流量的效率。

而不是：

视频数量。

17. 想防止 AI 过度分析，可以这样说

只分析到足以做决定为止；如果现实测试比继续分析更有价值，直接让我去测试。

这是 Thinking OS 非常核心的一句话。

18. 最推荐的复杂问题 Prompt

以后碰到真正重要的问题，你甚至可以固定使用：

`/deep`

我现在要解决：【问题】

真正目标：【目标】

已知事实：【事实】

我的判断/假设：【假设】

约束：【时间、资金、资源】

不要默认我的问题和方案是正确的。

先判断真正的问题是什么。

必要时寻找 Source of Truth。

区分事实、假设和未知。

只使用真正有价值的思维方法。

如果有明显风险，做 Red Team。

如果继续分析价值已经不高，停止分析。
最后给我：明确建议、关键依据、最大风险和下一步行动。

这是“高强度版万能模板”。

但仍然不需要每次使用。

19. Thinking OS 适合处理什么？

特别适合：

决策

做还是不做？

A 还是 B？

什么时候应该停止？

战略

未来半年重点做什么？

哪条业务线最重要？

商业

这个模式能不能赚钱？

第一阶段应该验证什么？

投资

当前 Thesis 还成立吗？

什么证据会让我改变判断？

产品

应该做 App 还是网页？

这个功能真的需要吗？

Coding

重构还是继续？

真正 Bug 在哪？

应该先修什么？

Research

这些证据到底够不够？

哪个 Source 才是真正可信的？

内容 / 运营

数据下降真正原因是什么？

活动机制到底哪里有问题？

长期项目

一个月前的判断和现在发生了什么变化？

应该继续、Pivot 还是 Kill？

20. 什么事情不需要 Thinking OS 深度工作？

例如：

帮我翻译这句话。

改一下文案。

算 3×8 。

给我10个标题。

把这句话写专业一点。

这种问题应该：

直接做。

如果 Thinking OS 对这种问题也开始：

“我们首先从第一性原理来看……”

那反而说明它工作错了。

21. Thinking OS 不应该做什么？

Thinking OS 不应该：

1. 为了显得聪明而套框架；
 2. 每个问题都 Red Team；
 3. 每个问题都搜资料；
 4. 把猜测说成事实；
 5. 无限研究而不行动；
 6. 因为你提出了某个方案，就默认方案正确；
 7. 为了复杂而复杂；
 8. 用十个模型解决一个一分钟的问题；
 9. 把好结果等同于好决策；
 10. 把坏结果等同于坏决策。
-

22. 关于联网、文件、邮箱、日历、代码等工具

Thinking OS 自己不会凭空获得数据。

它的作用是判断：

当前真正需要去哪里找答案。

例如：

当前公开信息

查当前官方/公开来源。

你的会议

Calendar 才是 Source of Truth。

你的邮件

Email 才是 Source of Truth。

代码问题

Repo / CI 才是 Source of Truth。

你上传的 PDF

文件本身才是 Source of Truth。

如果当前 ChatGPT 环境具备对应工具，Thinking OS 应优先使用正确来源。

如果没有对应工具，就应该明确告诉你缺什么，而不是假装知道。

23. 关于长期记忆

Thinking OS 有 Decision State 设计。

但是：

Decision State $\neq$ 自动永久记忆。

是否能跨聊天保存，取决于你当前使用的环境是否真的提供持久存储。

所以重要长期项目建议：

- 保留 Decision State；
- 放在项目文件 / Repo / 文档中；
- 下一次继续时加载回来。

不要仅仅依赖：

“AI 应该还记得。”

24. 推荐的日常使用流程

以后你可以完全按照这个流程：

第一步

正常提问。

我现在想做 XXX，你帮我看看。

第二步

如果发现这是重要问题：

`/deep`

第三步

如果你自己已经很看好：

`/red`

第四步

如果资料真实性很重要：

`/data`

第五步

决定已经做完：

`/action`

第六步

长期项目出现新数据：

`/update`

第七步

阶段结束：

`/review`

就是这么简单。

25. 最常用快捷指令速查表

指令	作用	什么时候用
<code>/simple</code>	简单处理	小事、可逆决定
<code>/fast</code>	快速结论	时间紧
<code>/deep</code>	深度分析	重要复杂问题
<code>/max</code>	合理范围最大深度	极重要问题

指令	作用	什么时候用
<code>/red</code>	红队攻击	防确认偏误
<code>/first</code>	第一性原理	重构问题
<code>/data</code>	提高证据要求	投资/研究
<code>/idea</code>	扩散探索	创意/方向
<code>/action</code>	停止讨论进入执行	已经决定做
<code>/state</code>	看当前长期状态	长期项目
<code>/update</code>	根据新证据更新	有新数据
<code>/review</code>	正式复盘	阶段结束
<code>/killcheck</code>	判断是否应该停止	项目生死判断
<code>/journal</code>	看历史判断变化	复盘

26. 10 个可以直接复制使用的提示词

① 判断项目值不值得做

我有一个项目：【描述】。真正目标是【目标】，预算【预算】，时间【时间】。帮我判断真正的问题是什么、值不值得做，以及最低成本应该先验证什么。不要默认我的方案正确。

② 深度决策

`/deep` 我准备做【事情】，失败成本比较高。帮我从概率、机会成本、可逆性、关键假设和最坏情况完整判断，最后给明确建议。

③ 红队

`/red` 这是我的方案：【方案】。先把它当成一个很强的方案，然后从最强反对者角度找出最可能失败的地方。

④ 第一性原理

`/first` 不考虑行业惯例，从目标、资源和底层约束重新分析【问题】。

⑤ 找真正原因

现在出现【问题】。已知事实是【事实】。不要直接猜原因，先给最可能的原因排序，再告诉我哪个测试最能区分它们。

⑥ 做执行计划

`/action` 方向已经确定。目标是【目标】，期限【时间】，资源【资源】。直接给执行模块、优先级、依赖、验收标准和第一步。

⑦ 查资料

`/data` 我要判断【问题】。优先找最接近 Source of Truth 的当前资料，区分事实、推断和未知，最后告诉我这些证据对决策意味着什么。

⑧ 防止过度工程

帮我看看这个方案是不是过度工程。如果能用更小、更便宜、更可逆的方法先验证，请优先给那个方案。

⑨ 长期项目更新

`/update` 新证据是：【数据】。结合我们之前的判断，告诉我哪些假设被加强或削弱、概率应该怎么变化，以及现在应该继续、修改还是停止。

⑩ 正式复盘

`/review` 重新审视这个项目。不要因为我们已经投入很多就倾向继续。根据当前事实重新判断 Thesis、关键假设、机会成本、Kill Criteria 和下一步。

27. 最终使用原则

以后使用 Thinking OS，只需要记住：

简单事情，直接问。

复杂事情，说清目标和约束。

重要事情，加 `/deep`。

怕自己上头，加 `/red`。

需要证据，加 `/data`。

已经决定了，加 `/action`。

长期项目有新数据，加 `/update`。

除此之外：

让 Thinking OS 自己工作。

你不需要替它决定使用什么思维模型。

28. 一句话版说明书

如果以后你什么都忘了，只记住这一句：

把你的真实问题、真实目标、真实约束告诉我，然后正常说：“用 Thinking OS 帮我判断一下；不要默认我的问题是对的，不要过度分析，最后给明确结论和下一步。”

这就够用了。